TECHNOVA SOLUTIONS PVT. LTD.

DEPLOYMENT POLICY

Document ID	TN-ENG-POL-005
Document Type	Engineering Deployment Policy
Document Owner	Engineering / Platform
Approving Authority	VP Engineering / CTO
Classification	Internal — Engineering
Version	1.0
Effective Date	01 April 2026
Review Cycle	Annual
Status	Active
Supersedes	New Policy

1. Purpose

This Deployment Policy establishes the standard requirements for moving software, configuration, infrastructure, data-model changes, and related engineering artifacts into test, staging, and production environments at TechNova Solutions Pvt. Ltd. (“TechNova” or “the Company”).

The objective is to make deployments controlled, traceable, repeatable, secure, and recoverable while supporting reliable delivery.

2. Scope

This policy applies to Engineering teams and authorized personnel responsible for deploying or operating Company applications, services, APIs, databases, infrastructure, AI/ML systems, automation, and configuration.

Project-specific deployment procedures may add controls based on system criticality, but may not bypass applicable security, access, review, or approval requirements.

3. Deployment Principles

- Deploy only changes that have an identified purpose and owner.
- Use approved source-control and deployment mechanisms.
- Maintain traceability between source revision, build artifact, and deployment.
- Validate changes before production deployment.
- Use least privilege for deployment access.
- Prefer repeatable and automated deployments.
- Plan rollback or recovery for changes where appropriate.
- Monitor the system after deployment.

4. Deployment Environments

Environment	Purpose
Development	Engineer-controlled development and early validation.

Environment	Purpose
Test	Automated and manual validation of integrated changes.
Staging	Production-like validation and release readiness where applicable.
Production	Live environment supporting business or customer operations.

5. Deployment Ownership

Each production service or deployment target should have a clearly identified owning team.

- Understand the deployment process and dependencies.
- Maintain appropriate deployment documentation.
- Ensure required checks and approvals are completed.
- Monitor deployments and respond to deployment-related incidents.

6. Deployment Authorization

Production deployments must be performed only by authorized personnel or approved automation.

- Deployment permissions must follow the Access Control Policy.
- Privileged deployment credentials must be protected.
- Users must not bypass deployment controls using shared credentials.
- Administrative overrides must be traceable where technically feasible.

7. Change Readiness

Before deployment, the responsible team should confirm that the change is sufficiently ready for the target environment.

- Required code review is complete.
- Required automated checks have passed.
- Known defects and risks are understood.
- Required configuration is available.
- Database or infrastructure migrations have been assessed.
- Monitoring and operational requirements are considered.

8. Build and Artifact Management

Production deployments should use a controlled build artifact or equivalent immutable release output where practical.

- Identify the source revision used to create the artifact.
- Use approved build systems.
- Protect build outputs from unauthorized modification.
- Keep release artifacts identifiable and reproducible where feasible.
- Do not manually alter production artifacts without an approved process.

9. Versioning and Release Identification

Production releases should have a unique and traceable version, tag, release identifier, or equivalent.

The identifier should allow the team to determine what source and configuration were intended for the deployment.

10. Deployment Checklist

- Change scope confirmed.
- Review and required approvals completed.
- Automated checks successful.
- Artifact or release version identified.
- Configuration validated.
- Migration requirements assessed.
- Rollback or recovery approach considered.
- Deployment window and stakeholders confirmed where applicable.
- Monitoring available.

11. Standard Deployment Process

- Prepare the approved change.
- Build and validate the release artifact.
- Deploy to the appropriate lower environment.
- Run required tests and validation.
- Promote the approved release through the deployment process.
- Verify production health.
- Record the deployment outcome.

12. Automated Deployments

Automation should be used where practical to reduce manual errors and improve consistency.

- Protect pipeline credentials and deployment secrets.
- Restrict who can modify production pipelines.
- Review material pipeline changes.
- Ensure deployment steps are reproducible.
- Provide appropriate logs and status information.

13. Manual Deployments

Manual production deployment may be used where automation is not practical or during controlled emergency procedures.

Manual deployments must still use authorized access, approved change records, validation, and appropriate documentation.

14. Deployment Windows

Teams may define deployment windows for systems where timing affects business operations, customer impact, staffing, or recovery capability.

Critical changes outside a defined window should use the applicable approval or emergency process.

15. Database Changes

- Assess schema and data changes before deployment.

- Use approved migration mechanisms.
- Consider backward compatibility during rolling deployments.
- Assess locking, runtime, and data-volume impact.
- Validate backup and recovery requirements where appropriate.
- Do not make undocumented production database changes.

16. Configuration Changes

Configuration changes can affect system behavior as significantly as code changes and must be controlled accordingly.

- Review material configuration changes.
- Protect secrets.
- Validate environment-specific values.
- Record important configuration changes.
- Avoid manual drift between environments.

17. Infrastructure Changes

Infrastructure changes should use approved infrastructure-as-code or controlled administrative processes where practical.

- Review security and access implications.
- Assess availability impact.
- Validate dependencies.
- Maintain rollback or recovery capability where appropriate.

18. AI and ML Deployments

AI and machine-learning systems require deployment controls appropriate to models, data, prompts, retrieval systems, and serving infrastructure.

- Validate the model or artifact being deployed.
- Confirm required data and access permissions.
- Protect model, API, and service credentials.
- Validate retrieval and authorization behavior where applicable.
- Monitor important quality, safety, performance, and operational signals.

19. Knowledge Assistant Deployments

Deployments affecting TechNova's enterprise knowledge assistant must preserve document and user authorization throughout ingestion, indexing, retrieval, context construction, and response generation.

- Test permission filtering before release.
- Verify restricted documents cannot enter unauthorized retrieval context.
- Validate citation and source-access behavior.
- Ensure permission changes are reflected in the retrieval layer.
- Verify indexing changes do not accidentally broaden access.

20. Security Checks

- Run required vulnerability and dependency checks.
- Protect deployment credentials.
- Verify access-control changes.
- Review exposed endpoints and configuration.
- Address critical security findings before production unless an authorized risk decision exists.

21. Testing Before Production

Production deployment should occur only after the required level of testing has been completed.

- Unit and integration checks as applicable.
- Functional validation.
- Regression validation.
- Security checks where required.
- Performance or load validation for relevant changes.
- Smoke or health checks.

22. Deployment Approval

Production releases may require approval from the responsible engineering owner, product owner, change authority, or another designated approver depending on the system and change risk.

Approval requirements should be defined by the team or service's operating model.

23. High-Risk Changes

Changes that could materially affect availability, security, data integrity, financial processing, customer experience, or critical infrastructure should receive additional review and planning.

- Identify impact and dependencies.
- Define validation criteria.
- Plan communication where appropriate.
- Prepare rollback or recovery steps.
- Use an appropriate deployment window.

24. Low-Risk Changes

Routine low-risk changes may use a streamlined deployment process when the service has an approved standard procedure and automated safeguards.

A streamlined process must not remove required security or authorization controls.

25. Canary and Gradual Rollouts

Where supported, teams may use canary, phased, percentage-based, feature-flagged, or other gradual rollout techniques for higher-risk changes.

- Define success and failure indicators.
- Monitor during rollout.
- Stop or roll back when predefined risk thresholds are reached.
- Expand deployment only after acceptable validation.

26. Feature Flags

Feature flags may separate deployment from feature activation.

- Restrict sensitive flags appropriately.
- Define flag ownership.
- Document intended behavior.
- Remove obsolete flags when safe.
- Do not assume a disabled feature eliminates all underlying security or operational risk.

27. Rollback

A rollback returns a service or component to a known acceptable state after an unsuccessful deployment.

- Define rollback conditions where practical.
- Maintain identifiable previous releases.
- Consider database compatibility before rollback.
- Verify system health after rollback.
- Record the outcome.

28. Recovery When Rollback Is Not Possible

Some changes, particularly irreversible data migrations, cannot be safely reversed. Teams must plan compensating recovery or remediation procedures before deployment when this risk exists.

29. Deployment Failure

- Stop further rollout where appropriate.
- Assess user and system impact.
- Communicate through approved incident channels.
- Rollback or remediate according to the planned procedure.
- Monitor recovery.
- Document significant failures and lessons learned.

30. Emergency Deployment

An emergency deployment may be performed when delay creates unacceptable operational, security, or business risk.

- Use the smallest safe change.
- Obtain expedited authorization where feasible.
- Perform the highest-priority available validation.
- Document the reason and actions.
- Perform retrospective review after stabilization.

31. Hotfixes

Hotfixes follow the approved Git Workflow and should remain narrowly scoped to the urgent production problem.

A hotfix should be integrated into relevant development or release branches after production stabilization where required.

32. Deployment Communication

For material changes, stakeholders should be informed of expected impact, timing, maintenance requirements, or user-facing changes where appropriate.

Communication should be proportionate to the change and system criticality.

33. Maintenance and Downtime

Where deployment may cause downtime or degraded service, teams should assess whether planned maintenance communication is required and select an appropriate window.

34. Monitoring After Deployment

- Check service health.
- Review error rates and important metrics.
- Check logs and alerts.
- Validate key user or business flows.
- Monitor dependencies.
- Continue observation for an appropriate period for the change risk.

35. Deployment Verification

A deployment is not considered complete merely because the deployment command succeeded. The responsible team should verify that the application or service is operating as intended.

36. Deployment Records

Deployment records should identify sufficient information to support operational traceability.

- Application or service
- Release/version identifier
- Source revision where applicable
- Deployment time
- Environment
- Deployment outcome
- Relevant approver or automation record

37. Auditability

Production deployment activity should be attributable to an authorized individual or approved automation identity.

Logs and deployment records should be protected according to applicable security and retention requirements.

38. Separation of Duties

For sensitive systems, development, approval, deployment, and production administration responsibilities should be separated where practical.

Automated deployment may provide controlled separation when permissions and pipeline ownership are appropriately designed.

39. Access During Deployment

Engineers must use only the permissions required for deployment and operational activities.

Temporary elevated access should expire or be revoked when the need ends.

40. Secrets During Deployment

- Use approved secret-management mechanisms.
- Do not print secrets into build or deployment logs.
- Do not store credentials in deployment scripts.
- Rotate compromised deployment credentials promptly.

41. Environment Parity

Teams should keep environments sufficiently similar to identify meaningful deployment problems before production where practical.

Differences between environments should be documented when they are intentional and operationally relevant.

42. Dependency Changes

Dependency upgrades should be tested and reviewed for compatibility, security, and operational impact before production deployment.

Major dependency changes may require additional validation or staged rollout.

43. Deployment of Documentation

Documentation changes affecting operations, APIs, configuration, or user behavior should be deployed or published consistently with the associated software change.

44. Ownership After Deployment

The owning team remains responsible for monitoring and addressing issues introduced by its deployments.

Deployment completion does not transfer operational responsibility to another team unless a documented ownership transition occurs.

45. Incident Escalation

If a deployment causes or contributes to a significant incident, the team must follow the applicable Incident Response process.

Engineering should preserve relevant deployment and system evidence for investigation.

46. Post-Deployment Review

Significant or problematic deployments may require a post-deployment review.

- Was the change delivered as intended?
- Did monitoring identify issues promptly?
- Was rollback or recovery effective?
- Are follow-up actions required?
- Should the deployment process be improved?

47. Deployment Anti-Patterns

- Deploying unreviewed production code.

- Using shared privileged credentials.
- Making undocumented production changes.
- Deploying directly from an engineer's untracked local environment.
- Ignoring failed automated checks without authorization.
- Deploying without considering database compatibility.
- Disabling monitoring to hide deployment errors.
- Continuing a rollout after predefined failure indicators are reached.

48. Exceptions

Exceptions to this policy must be documented, justified, risk-assessed, and approved by an authorized owner.

Emergency situations may use the emergency deployment process but should receive retrospective documentation and review.

49. Responsibilities

Role	Responsibilities
Engineer	Prepare changes, complete required validation, and follow deployment procedures.
Reviewer / Approver	Evaluate readiness and approve according to the required level of authority.
Service Owner	Ensure deployment readiness, operational ownership, monitoring, and recovery capability.
Platform / IT	Maintain deployment infrastructure, access controls, and supporting automation.
Information Security	Provide security requirements and support assessment of security-sensitive deployments.

50. Frequently Asked Questions

Can I deploy directly from my laptop?

Production deployments should use approved controlled mechanisms so the deployed version is traceable and reproducible.

Does a successful deployment command mean the release is successful?

No. The service should also be verified through appropriate health, functional, monitoring, and operational checks.

Can an emergency deployment skip normal approval?

Emergency changes may use expedited authorization when delay creates unacceptable risk, but they remain subject to traceability and retrospective review.

Do configuration changes need review?

Material configuration changes should be controlled like other production-impacting changes.

What if a database migration cannot be rolled back?

Plan compensating recovery or remediation before deployment and assess compatibility carefully.

Should every deployment have a rollback?

A rollback or recovery approach should be considered; some changes require an alternative remediation plan because they cannot safely be reversed.

Who is responsible after deployment?

The owning engineering team remains responsible unless a documented ownership transition occurs.

Can feature flags replace testing?

No. Feature flags control activation but do not remove the need for appropriate testing and security validation.

What should I do if deployment causes an incident?

Stop or contain the rollout as appropriate, follow incident procedures, communicate impact, and use rollback or remediation steps.

Where are Git branching and hotfix rules defined?

In the approved Git Workflow document.

51. Policy Administration

Engineering owns this Deployment Policy and reviews it at least annually or when material changes occur in deployment tooling, infrastructure, security requirements, release practices, or organizational needs.

The latest approved version maintained in the official Company repository is the authoritative version.

52. Revision History

Version	Effective Date	Description	Owner	Approval
1.0	01 April 2026	Initial release of Deployment Policy	Engineering / Platform	VP Engineering / CTO

53. Related Documents

- Engineering Handbook
- Coding Standards
- Git Workflow
- Code Review Guidelines
- Testing Standards
- Information Security Policy
- Access Control Policy
- Incident Response
- Business Continuity

END OF DOCUMENT

TN-ENG-POL-005 | Version 1.0 | Internal — Engineering | Engineering / Platform